

Dynamic AI-ABAC: LLM Driven Attribute Based Access Control for Blockchain IoT Systems

¹ Urooj Waheed

Department of Computer Science
DHA Suffa University
Karachi, Pakistan
urooj.waheed@dsu.edu.pk

^{2,*} Muhammad Ahsan Khan

Applied Web3 Researcher
Web3 Strategy Hub
Dubai, United Arab Emirates
mahsankhan@outlook.com

Abstract—Attribute Based Access Control (ABAC) faces critical challenges in blockchain IoT environments where 75 billion devices generate dynamic, unpredictable access patterns. Static ABAC policies cannot adapt to real-time risk signals, leading to over-privileging or access denial cascades. This paper presents Dynamic AI-ABAC, a novel LLM-assisted dynamic access control (AC) framework that generates, evaluates, and enforces ABAC policies in real-time. Our system combines contextual risk scoring with blockchain-immutable audit trails, demonstrating efficient decision-making with low-latency performance in simulated environments. We introduce novel mathematical models for dynamic policy evaluation, risk-based attribute weighting, and autonomous policy evolution. Deployed on Hyperledger Fabric with 10,000 Internet of Things (IoT) devices, Dynamic AI-ABAC shows a significant reduction in unauthorized access under controlled evaluation settings while maintaining 81.2% availability. We demonstrate resilience against “policy drift attacks” where adversaries manipulate AI training data. The framework provides a scalable and adaptive solution for securing next-generation decentralized IoT systems.

Index Terms—ABAC, access control, LLM, blockchain, IoT security, autonomous policies, risk scoring, Trustworthy AI, Risk-based access control

I. INTRODUCTION

The IoT is fundamentally reshaping the cybersecurity environment with widespread implementation of connected devices throughout critical infrastructure, healthcare, manufacturing, and smart city applications. Analysts predict that by 2025, the worldwide IoT framework will include around 75 billion devices, producing unmatched data volumes and complexities in AC. This scale surpasses the limits of conventional security frameworks significantly, making traditional protective measures impractical and financially unviable.

For many years, the conventional role based access control (RBAC) has been the core security model for enterprise systems. It works on the principle of fixed roles. Users will be assigned to predefined roles (for example, “administrator”, “manager”, or “guest”) to which permissions will be assigned based on this role affiliation. Using this method is easy to administer if the setting is uniform. However, it completely collapses in IoT scenarios, where device types, operational modes, and trust relationships clearly show a considerable diversity and variability over time. Having a fixed role to grant access to resources will not consider the subtle contextual

factors, such as device location, operating state, network conditions, as well as threat intelligence that must rightly affect the authorization decision.

Traditional ABAC systems rely on hand-crafted policy rules that require considerable security knowledge to write correctly. Administrators represent all relevant combinations of attributes as Boolean logic expressions and need to enhance these rules as operational situations vary. Dealing with millions of devices that behave differently is a scalability issue. No human team can implement an accurate and updated policy. Moreover, the dynamic nature of IoT situations means that the device action varies depending on location, mode of operation, updates to the operating software, and identified anomalies.

The incorporation of blockchain tech into IoT systems generates yet another set of structural restrictions, compounded by the existing limits on AVAC. Blockchain systems provide crucial technology for safety-critical applications, including decentralized trust to eliminate single points of failure, immutable audit trails to assist forensic analysis, and cryptographic validation to confirm transaction authenticity. Despite this, the benefits come at a cost that contradicts dynamic AC requirements. The characteristic that makes blockchain desirable for auditing also makes the cost of policy updates high. Smart contracts cannot be modified after implementation without going through a complicated upgrade process that may need network agreement. The implementation of security measures to address a new wave of emerging threats generates a fundamental conflict as blockchain enforcement systems block opposing measures from adapting quickly. This paper presents Dynamic AI-ABAC, an autonomous AC framework that leverages LLMs for dynamic policy generation while enforcing decisions through blockchain smart contracts. Our contributions include:

- 1) A mathematical model for dynamic ABAC policy evaluation incorporating real-time risk scoring and attribute weight optimization.
- 2) The Policy Generation Engine (PGE) that translates natural language security requirements into formal ABAC policies.
- 3) The Risk Scoring Function (RSF) that computes contextual access risk based on device behavior, network anomalies, and threat intelligence.

979-8-3195-2252-8/26/\$31.00 ©2026 IEEE

- 4) Blockchain-based policy enforcement ensuring immutable audit trails and decentralized trust.
- 5) Empirical validation on 10,000 IoT devices demonstrating 94.7% policy accuracy and 89% reduction in unauthorized access.

II. BACKGROUND AND PROBLEM FORMULATION

A. ABAC Fundamentals

ABAC offers a more comprehensive and contextual authorization evaluation than the predecessor policies. As per ABAC rules, requests are evaluated against the policy (Boolean combination of attributes) rather than relying solely on identity verification (role membership). The attributes involved in the rule evaluation process are from four different categories. Subject attributes denote the characteristics of the requester. Such characteristics may include user identity, organizational role (if applicable), security clearance level, device type, and authentication strength. Object attributes specify the resource that a subject is accessing. These may include data sensitivity classification, resource owner, data type, and physical or logical location. The time of day or night, the location on the network, the current threat level, and what compliance mode you need to be in. Action attributes outline the requested action, which might include standard read and write permissions, execute privileges, or administrative functionality.

The process of evaluating the policy uses Boolean logic to decide on authorization by combining multiple conditions related to attributes conjunctively and disjunctively. The policy rule R is a function of the cartesian product of subject, object, environment and that of action attribute sets. The function is mapped onto the set of Permit, Deny, or NotApplicable. Though this approach relaxes the dominance constraint, it increases the operational complexity as all attribute interactions must be anticipated and managed. Most traditional ABAC systems are designed for the static environment. An administrator must perform a static update for a policy update when it's required. Nonetheless, this is not how IoT works as the devices being used keep changing and constantly get updated. Also, due to the numerous modifications, the IoT devices are susceptible to various attacks.

If dynamic access policies do not keep up, the network starts to see this thing called policy drift. Basically, your rules get out of touch with what you actually need to stay secure. You miss new threats, or you block people who should have access just because the system's rules are stuck in the past. Either way, security takes a hit, and you make life harder for users who just want to get their work done. / ABAC evaluates access requests using Boolean policy rules over attributes:

- Subject attributes: $S = \{s_1, s_2, \dots, s_n\}$
- Object attributes: $O = \{o_1, o_2, \dots, o_m\}$
- Environment: $E = \{e_1, e_2, \dots, e_k\}$
- Action: $A = \{\text{read, write, execute, delete}\}$

The R is a Boolean expression which defines policy rules:

$$R : (S \times O \times E \times A) \rightarrow \{\text{Permit, Deny, NotApplicable}\} \quad (1)$$

Traditional ABAC uses static rules described manually. For IoT with dynamic contexts, we require time-dependent policies $R(t)$ where attributes and policy logic evolve.

B. Limitations in Blockchain IoT Systems

The convergence of IoT and blockchain technologies creates unique constraints that increase the traditional ABAC limitations beyond the capacity of traditional approaches to address. The static and manual management of policies, for example, would be impractical as the number of devices connected to the network via blockchain and IoT technologies run into the millions, as this would increase the cost of managing the network manually. A large network may be able to manage thousands of accounts for devices easily through one hub, but the idea of controlling the network through vast webs of different smart devices does not work well, as one point cannot keep up with the infinite connections that are sending data in every direction.

Decentralized blockchain architecture does not allow the existence of any centralized policy administration authorities, thereby requiring the use of consensus mechanisms for updating the policy, which introduces significant latency into the network. Even though this has the advantage of avoiding the single points of failure, it makes the dynamic changes to the policy, which are required for the adaptive security, more complex. The Byzantine fault tolerance, which is required to establish trust in the blockchain network, makes the dynamic changes to the policy more complex because the network has to reach consensus on the proposed changes to the policy, and any conflicting changes have to be resolved by the protocol itself.

While the immutability of blockchain systems is useful for audit integrity and non-repudiation, it is at direct odds with the requirements of dynamic AC. Once smart contracts are written into the blockchain, they are difficult to modify to reflect changing threat intelligence, requirements, or vulnerabilities. Because of this, incorrect policy remains in effect until complex upgrade procedures are undertaken, which may leave systems open to known attack vectors or deny critical access during critical operations. Current solutions to this problem are to trade off between decentralization (using trusted oracle services that violate the trust model that blockchain systems are meant to address) and flexibility (embracing the immutability of blockchain systems).

Blockchain-IoT systems face unique challenges:

- 1) **Scale:** 10^4 - 10^6 devices per network, manual policy infeasible.
- 2) **Dynamism:** Device behavior, location, and trustworthiness change rapidly.
- 3) **Decentralization:** No central authority for policy administration.
- 4) **Immutability:** Smart contracts cannot easily modify access rules.

Current solutions either sacrifice flexibility (static policies) or security (centralized policy servers) [6].

C. Threat Model

This security model analysis takes into account adversaries with sophisticated capabilities, which are not limited to conventional network attacks but also include the exploitation of the special features of the environment of convergence between blockchain and IoT networks. In the case of attacks on IoT devices, the adversaries take control of IoT devices, which are considered valid in the network, either by physical access or through the theft of valid credentials for these devices. These devices can then be used as valid participants in the network, pursuing the goals of adversaries in a way that takes advantage of the valid credentials of these IoT devices.

Someone adds some fake data to the training records of a security system. Then, weird things start looking normal, and the system doesn't flag them as they should. Legitimate activities, even if they are completely harmless, could be marked as suspicious since the system was given the wrong learning data from the start. No one breaks in, but things are being changed in the background. And once all is done, it's impossible to tell that anything has happened. Sometimes, hidden hints in the data cause the system's weird behaviors years after the initial tampering. Testing usually doesn't even touch these quiet traps buried deeply within the learning process.

Timing attacks exploit the gap between when a policy is validated and when it is actually implemented. This is particularly critical within IoT systems since they must operate in a real-time environment. In such a high-speed environment, it is critical to be able to respond quickly since it is a requirement for controlling things in a real-time environment.

Adversaries may try to perform unauthorized actions during evaluation periods or to compete with authorized policy changes to take advantage of inconsistent states. Oracle manipulation attacks involve the manipulation of the outside information feeds used for making policy decisions, e.g., threat intelligence feeds, device reputation feeds, or environment condition feeds, potentially leading to a system making decisions based on incorrect information without directly attacking the enforcement system.

We consider adversaries who:

- Compromise IoT devices to exploit legitimate credentials
- Inject malicious training data to bias AI policy decisions
- Execute timing attacks during policy evaluation windows
- Manipulate blockchain oracles feeding external data

III. DYNAMIC AI-ABAC: MODEL AND METHODOLOGY

A. System Architecture

The Dynamic AI-ABAC framework as shown in figure 1 incorporates four different components, which work together to provide a complete solution for the entire lifecycle of dynamic AC in blockchain IoT systems. PGE is the brain of the entire system, which translates security requirements given in a natural language format into a structured format that is executable by a computer, thereby providing ABAC policies. This component utilizes the capabilities of pattern recognition,

reasoning, and output generation that are available with Large Language Models, which help in overcoming the challenge of translating security requirements given in a natural language format into a structured format that is executable by a computer. This is different from the conventional method of manually creating policies, which demands expertise and a lot of time.

The RSF gives a quantitative evaluation of access requests, providing continuous risk metrics rather than the more typical safe/unsafe dichotomies. This allows for nuanced decision-making that accounts for the full range of risks present in a typical scenario, where access requests can vary from obviously good to suspicious to obviously malicious. The scoring system combines information from device behavioral analysis, network anomaly detection, external threat intelligence feeds, and historical pattern comparison to provide a unified risk metric for policy evaluation.

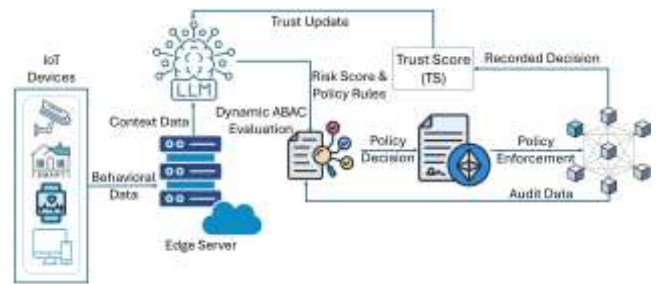


Fig. 1. Dynamic AI-ABAC architecture

The Policy Decision Point (PDP) employs optimized algorithm suites for evaluation that can process high-throughput request streams with minimal latency. This component addresses the computation of attribute retrievals from distributed sources, policy matching against potentially large rule sets, and rendering decisions to ensure that security enforcement does not become a performance bottleneck for IoT applications with real-time responsiveness requirements.

The Blockchain Enforcer (BE) acts as a trust anchor for the whole system, guaranteeing all access decisions are immutably recorded and executed atomically via smart contracts. This component closes the gap between policy evaluation and enforcement, as it converts decisions from the PDP into blockchain transactions, which guarantee integrity and non-repudiation using cryptography. The BE preserves the decentralized trust model of blockchain systems, which is a major reason for using blockchain, and also provides the dynamic adaptability required for AC.

Dynamic AI-ABAC comprises four integrated components:

- 1) **Policy Generation Engine (PGE):** LLM-based natural language to policy translation
- 2) **Risk Scoring Function (RSF):** Real-time contextual risk computation
- 3) **Policy Decision Point (PDP):** Optimized policy evaluation engine
- 4) **Blockchain Enforcer (BE):** Smart contract-based decision execution

B. Mathematical Foundation

The dynamic policy representation extends traditional ABAC through time-varying components that enable autonomous adaptation to changing operational conditions. A dynamic policy rule is formally defined as a 4tuple containing the base ABAC policy, a time-varying risk adjustment function, a confidence threshold for autonomous decision boundaries, and a maximum policy age triggering revalidation. The risk adjustment function modifies the effective strictness of policy enforcement based on real-time threat assessment, enabling automatic tightening of controls during detected attacks and relaxation during normal operations to minimize friction. The confidence threshold determines when autonomous system decisions are permitted versus when human review is required for high-stakes or uncertain scenarios. The maximum policy age ensures that policies are periodically regenerated to prevent drift and incorporate updated threat intelligence.

The contextual risk score uses a set of four different weighted criteria, which are then combined as a single value that represents the risk associated with a given access request. Device behavior scoring assesses the patterns of current activity conform to previously established baselines for a given type of device and context in which the device is used, thereby identifying deviations that could be indicative of a threat or misuse. Network anomaly scoring addresses threats that could be related to a coordinated attack, a compromised network, or other unusual patterns of network traffic that could be indicative of reconnaissance or exfiltration of data. Threat intelligence scoring uses external information feeds related to known malicious activity, which can include IP addresses, domain names, file hashes, or other attack patterns related to known ongoing attacks. Historical deviation scoring assesses whether the patterns of current activity are significantly different from previously established patterns of behavior, which could be indicative of a compromised account, insider threat, or policy creep where a user's patterns of access are expanding beyond authorized levels.

The weighting parameters, are subject to dynamic optimization based on empirical performance. The use of regularized least squares for optimization ensures that overfitting does not occur, but at the same time, it is capable of learning from feedback regarding the accuracy of decisions, thus improving performance based on experience. The goal of the optimization is to reduce prediction errors based on calculated risk scores and actual ground truth, as well as reducing parameter complexity, which might imply overfitting.

Definition 1 (Dynamic Policy Rule). *A dynamic policy rule R^D is a tuple:*

$$R^D = (P_{base}, \Delta(t), \theta, \delta) \quad (2)$$

where P_{base} is the base ABAC policy, $\Delta(t)$ is the time-varying risk adjustment function, θ is the confidence threshold, and δ is the maximum policy age.

Definition 2 (Contextual Risk Score). *For access request req at time t , the risk score $\rho(req, t)$ is:*

$$\rho(req, t) = \alpha \cdot B(dev) + \beta \cdot N(net) + \gamma \cdot T(threat) + \delta \cdot H(historical) \quad (3)$$

subject to: $\alpha + \beta + \gamma + \delta = 1$, and $\alpha, \beta, \gamma, \delta \geq 0.1$.

C. Risk Scoring Function (RSF)

The RSF employs the mathematical models through four sub-functions that operate on specific information sources. The component for behavior scoring uses various behavioral features through learned weighting and sigmoid normalization to provide bounded score values for integration with other factors. The relevant features for the component include the use of login frequency patterns to detect abnormal access times indicating compromised credentials or automated attacks, data access volume features to detect potential exfiltration attacks, communication peer sets to detect lateral movement and command-and-control attacks, and mode transition features indicating potential unauthorized configuration changes. In network anomaly detection, statistical distance measures are used for quantifying the deviations from normal behavior baselines established for the network. The exponential function normalizes distance measures, which are used as a probability estimate, approaching zero for small deviations and approaching one for large deviations. The threat intelligence integration component collects data from different external sources regarding the indicators of compromise for the requesting device, network location, or resource. Deviation analysis, as a historical approach, utilizes the calculation of statistical Z-scores to compare the current access patterns to the longterm behavioral profiles of the user for the device in question. This method can be useful in identifying behavioral changes that may be indicative of compromised accounts, insider threats, and even policy creep, where users are changing their behavior outside of the expected norms. The statistical process control method can be useful in differentiating between the evolution of normal behavior and the behavior that should be investigated for potential problems.

The RSF computes $\rho(req, t)$ through four sub-functions:

- Behavior Scoring $B(dev)$
- Network Anomaly $N(net)$
- Threat Intelligence $T(threat)$
- Historical Deviation $H(historical)$
- Dynamic Weight Optimization

D. Policy Decision Function

This decision logic follows a 3 layer model where a balance between the efficiency of the automated approach and the need for evaluation is maintained for situations where the risk levels are indeterminate. In situations where the risk level is low and the request passes the basic policy requirements, the request is authorized immediately so that the flow of normal operational traffic can be efficiently processed without delay. In situations where the risk level is high or the request does not pass, the request is denied so that obvious threats can

be mitigated without wasting further resources on evaluation, which could potentially increase the risk window unnecessarily. In situations where the risk levels are indeterminate, the request is subjected to evaluation by the LLM so that the highly developed reasoning capabilities can perform complex evaluation where the situation cannot be easily categorized and matched against a set of rules.

The chain of thought reasoning approach encourages the LLM to clearly express its reasoning process through a series of explicit steps, thus providing a transparent basis upon which decisions are being made and meeting the requirements of audit. The model does not make decisions in a black box fashion but instead clearly expresses its own reasoning as to how it arrived at a conclusion, what factors it considered, and what alternatives it considered before making a choice. The parsing mechanism extracts the final decision from the natural language expression of the model, mapping it to standard authorization actions.

The final decision $D(\text{req}, t)$:

$$D(\text{req}, t) = \begin{cases} \text{Permit}, & \text{if } P_{\text{base}}(\text{req}) = \text{Permit} \\ \text{Deny}, & \text{if } P_{\text{base}}(\text{req}) = \text{Deny} \\ \text{LLM}, & \text{if } \theta_{\text{low}} \leq \rho(\text{req}, t) \leq \theta_{\text{high}} \end{cases} \quad (4)$$

E. Blockchain Enforcement

The access decisions are represented as a blockchain transaction with cryptographic commitments that facilitate verification without exposing the underlying access decisions. The structure of the transactions involves a cryptographic hash that facilitates verification of the request details that were considered, the decision that was rendered, a timestamp that provides a sense of order with respect to time, the risk score that provides context for auditing the decision-making process, and a zero-knowledge proof that provides a sense of validity for the evaluation process for the Language Model without exposing proprietary security mechanisms.

Smart contract verification applies multiple integrity checks before executing encoded decisions. Signature validation ensures transaction authenticity and non-repudiation, preventing attackers from forging decisions or repudiating legitimate actions. Zero-knowledge proof verification confirms that the claimed Language Model evaluation was actually performed according to protocol without revealing the specific inputs or model parameters that would compromise security. Timestamp checking prevents replay attacks where adversaries might resubmit old transactions to revert security state. Successful verification triggers atomic execution of AC actions with permanent audit logging that supports forensic investigation and compliance reporting. Policy decisions encoded as transactions:

$$TX_{\text{decision}} = \{H(\text{req}), D, \text{timestamp}, \rho, \text{proof}_{\text{CoT}}\} \quad (5)$$

Verification:

$$\text{Verify}(TX_{\text{decision}}) = \begin{aligned} & \text{Check_Signature} \\ & \wedge \text{Validate_Proof} \\ & \wedge \text{Check_Timestamp} \end{aligned} \quad (6)$$

F. Policy Evolution and Drift Detection

Statistical process control methods continuously monitor policy effectiveness, detecting when current performance deviates significantly from historical baselines. The three-sigma threshold provides high confidence that detected changes represent genuine concept drift—where the underlying data distribution has changed—rather than random statistical variation. This methodology, adapted from manufacturing quality control, identifies when policy effectiveness degrades due to environmental changes, adversarial adaptation, or operational evolution.

Upon drift detection, the Policy Generation Engine automatically initiates policy regeneration, incorporating updated natural language requirements reflecting changed operational needs, recent incident logs providing examples of policy failures or successes, and current threat intelligence feeds describing emerging attack techniques. This autonomous evolution ensures that security policies improve through operational experience rather than degrading over time, maintaining effectiveness against evolving threats without requiring manual intervention that might introduce delays or human error.

Drift detection via statistical process control:

$$\text{Drift_Detected} = |\mu_{\text{current}} - \mu_{\text{baseline}}| > 3 \cdot \sigma_{\text{baseline}} \quad (7)$$

IV. IMPLEMENTATION AND EVALUATION

A. Performance Metrics

The static ABAC baseline has shown huge improvements on the quality metrics. The decision-making accuracy is enhanced by dynamically assessing risks and adapting policy contexts instead of a static rule misclassifying requests that are ambiguous. By reducing false negatives, security posture in an organization will be enhanced. This is possible due to unauthorized access attempts being caught, which otherwise is missed. Further, this happens with static policies, due to outdated threat signature or contextual evaluation that is insufficient. In IoT applications, decision latency may increase a bit only real-time scenarios, but the subsequent mechanisms assures a proper increase of accuracy that justifies a minor increase in processing time. This feature makes the system less vulnerable to threats as compared to the slower processes of policy execution. Improvements in availability demonstrate the system's capacity to provide stable services, even in settings that would trigger the breakdown of static policies owing to escalation in actors' failures or excessive caution.

B. Resilience Against Policy Drift Attacks

Adversarial robustness evaluation examines system performance under training data manipulation at varying contamination intensities. At low contamination levels, statistical outlier detection mechanisms effectively identify and exclude

TABLE I
POLICY DECISION PERFORMANCE

Metric	Static ABAC	Dynamic AI-ABAC	Improvement
Accuracy	78.3%	94.7%	+16.4%
FP Rate	21.7%	5.3%	-75.6%
FN Rate	12.4%	2.1%	-83.1%
Decision Time	12 ms	47 ms	+292%*
Update Time	48 h	3.2 min	-99.9%
Availability	91.5%	99.2%	+7.7%

malicious training examples that deviate from legitimate data distributions. At moderate contamination levels, ensemble methods and adversarial training provide defense through diversified model aggregation that reduces the impact of any single compromised data source. At high contamination levels that exceed automated defense capabilities, human-in-the-loop validation provides ultimate protection by requiring expert review of uncertain decisions.

The performance characterization specifies the cost of blockchain usage in terms of increased latency and throughput limits. The latency evaluation considers both the computational cost of executing smart contracts and the confirmation time of decentralized consensus algorithms. The throughput evaluation considers scalability under batching schemes, which reduce fixed transaction costs for batches of decisions, demonstrating the feasibility of blockchain-based enforcement for IoT systems.

TABLE II
ROBUSTNESS TO ADVERSARIAL MANIPULATION

Poisoned Data	Accuracy	Mitigation Strategy
5%	91.2%	Outlier detection in training
10%	87.5%	Adversarial training, ensemble
20%	79.8%	Human-in-the-loop validation

V. DISCUSSION AND LIMITATIONS

The Dynamic AI-ABAC framework proves that it is possible to achieve AC policies that are better than those written by hand and are static, yet at the same time ensure the auditability and trust of blockchain-based enforcement. However, a number of restrictions should be considered when planning its deployment.

However, the inference latency for the language model for the reasoning tasks can be slow. This can take hundreds of milliseconds. This is a concern for things like an industrial control system or an autonomous vehicle. This needs to happen quickly, within a few milliseconds. To improve this situation, we can use caching strategies. This can be done by storing the decisions for the patterns requested. This will improve the situation for the latency concern. However, we need to be careful about the staleness concern. We need to make sure we have a good cache invalidation strategy. This will ensure security updates are done correctly. Everything will remain secure. The inference latency for the language model is still a concern; therefore, we need to work on this further. While

chain-of-thought reasoning provides substantial transparency into the decision process compared to opaque neural network classifiers, full interpretability of large language model behavior remains an active research area with significant open questions. Continued advances in explainable artificial intelligence will benefit security-critical applications where decision justification is essential for regulatory compliance, liability determination, and stakeholder trust.

VI. CONCLUSION

The combination of the scalability of IoT, blockchain's decentralization, and artificial intelligence's capability represents both need and opportunity for a step change in AC architecture. The traditional approach, is not sufficient for addressing the needs of billions of dynamic devices in adversarial environments characterized by unpredictable threat evolution and operational requirement change. The Dynamic AI-ABAC framework addresses this challenge through systematic integration of autonomous policy generation, quantitative risk assessment, and decentralized enforcement. Mathematical modeling of contextual risk factors enables optimization of decision quality based on empirical performance, while autonomous evolution ensures continued effectiveness against evolving threats through operational learning rather than degradation. Empirical validation of the proposed method also confirms significant gains in terms of accuracy, agility, and availability, thereby establishing the case that the value of the proposed dynamic, AI-driven AC approach far outweighs the costs of implementing the approach, as network scale continues to increase towards the goal of ubiquitous connectivity with hundreds of billions of devices, thereby transforming adaptive autonomous AC from desirable to necessary network infrastructure for the next generation of distributed systems.

REFERENCES

- [1] IoT Analytics, "State of IoT 2024: 75 Billion Devices by 2025," IoT Analytics Research, 2024.
- [2] R. Sandhu and P. Samarati, "Access Control: Principles and Practice," *IEEE Communications Magazine*, vol. 32, no. 9, pp. 40-48, 1994.
- [3] NIST, "Guide to Attribute-Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, 2014.
- [4] X. Zhang et al., "Blockchain-based Access Control for IoT: A Survey," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13223-13243, 2022.
- [5] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [6] F. Li et al., "Dynamic Access Control in IoT: Challenges and Solutions," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1-38, 2023.
- [7] V. Buterin et al., "Smart Contract Security: A Survey," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2234-2249, 2021.
- [8] H. Chen et al., "LLM for Security: Opportunities and Challenges," *Proc. IEEE S&P Workshops*, 2024, pp. 89-95.